

Lab 2: Boolean Logic

Prepared by

Student name: Aryan Qureshi

Student ID: 300467938

Student name: Borna Chadegani

Student ID: 300488971

Group # 10

ITI 1100 Digital Systems I

Professor: Dr. Aneta Traikova, PhD, P. Eng.

Presented to:

TA and Lab Supervisor:

Dorsa Khodabakhsh (dkhod005@uottawa.ca)

Rahil Ghalamboland (rghal080@uottawa.ca)

Lauren Hendley (lhend093@uottawa.ca)



uOttawa

School of Electrical Engineering and Computer Science

École de science informatique et de génie électrique

Faculty of Engineering | Faculté de génie

Summary

The lab involved analyzing and optimizing a Boolean logic function that included several input variables using techniques of Boolean algebra. The starting basis was to analyze the logic function by deriving its truth table, which described the behavior of the output for all possible combinations of the inputs. Boolean identities and algebraic rules were then applied to simplify the expression and obtain a minimized form of the logic function.

The simplified logic was implemented using Quartus. The design process includes creating a block diagram representation, compiling the circuit, and generating simulation waveforms to observe and verify the input/output relationships. To validate further, the designed circuit was mapped onto the Altera DE2-115 board and checked with physical inputs and outputs to confirm that the hardware matched the expected theoretical and simulated results. This demonstrates the practical importance of Boolean simplification in reducing the complexity of the circuit without affecting the correctness of the functionality.

Introduction

Background

Boolean logic is the mathematical foundation of digital systems and is used to describe the behavior of logic circuits that operate on binary variables. In Boolean logic, variables can take on one of two possible values, 0 or 1. Logical relationships between these are expressed using Boolean operators, which form the basis of all combinational logic circuits. By combining these basic operators, complex logical functions can be represented and implemented in digital hardware.

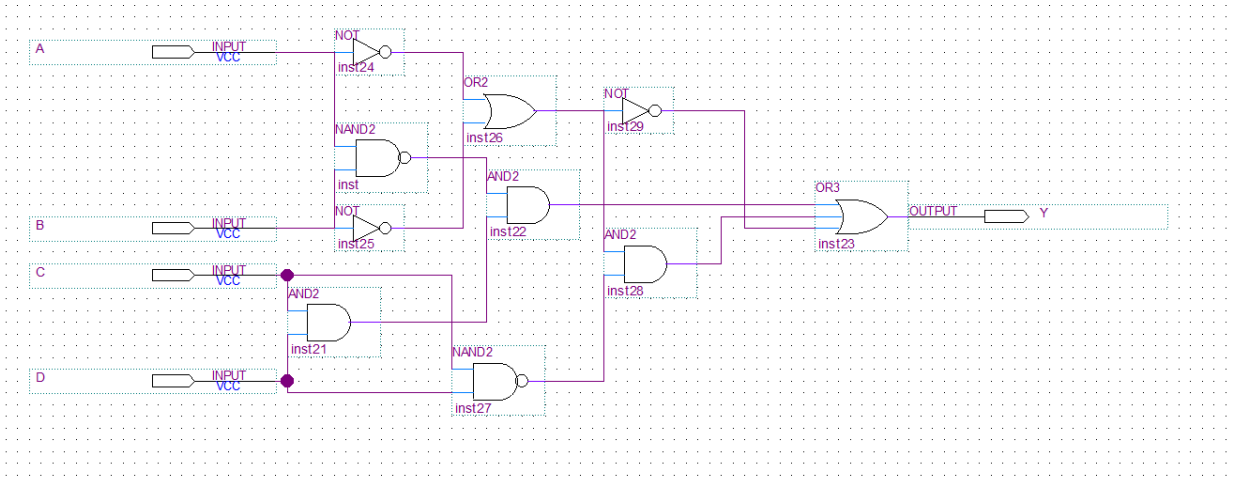
Objective of the Report

The primary objective of this report is to analyze and implement a minimized combinational logic circuit derived from a given Boolean expression. Specific goals of this experiment include:

- Deriving the truth table corresponding to a given Boolean logic function
- Simplifying the logic function using Boolean algebra identities
- Designing and implementing the minimized logic circuit using Quartus software
- Verifying the correctness of the minimized circuit through simulation waveforms
- Validating the circuit behavior through hardware testing on the Altera DE2-115 board

Pre Lab

The objective of the pre-lab was to draw the logic circuit that accurately represents the logic function shown below. Later this was compared to a truth table that had been created by theoretically analysing the logic formula and applying rules of boolean algebra to get the binary value for the output Y.



This circuit consists of 4 inputs A,B,C, and D. Inputs A and B are connected to the NAND2 logic gate initially then connected to AND2 from the AND2 of C and D, this forms the first input in the final OR3 gate. Second comes the middle expression which is the OR2 of A not and B not which goes into an AND2 where it is joined also by a NAND2 gate consisting of inputs C and D, this then forms the second input to the final OR3 gate. Lastly the alternate pathway of the OR2 of A not and B not is used with it also being NOT-ed and goes into the last input of the final OR3 gate, all of which is connected to the output Y.

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	1
0	0	1	1	1
0	1	0	0	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

and more combinations of inputs

$$Y = \overline{A} \overline{B} C D + (\overline{A} + \overline{B}) (\overline{C} D) + \overline{A} + \overline{B}$$

TruthtablePrelab2.PNG

$$Y = \overline{A} \overline{B} C D + (\overline{A} + \overline{B}) (\overline{C} D) + \overline{A} + \overline{B}$$

DeMorgan's Law: $Y = (\overline{A} + \overline{B}) C D + (\overline{A} + \overline{B}) (\overline{C} D) + \overline{A} + \overline{B}$

$$\rightarrow Y = (\overline{A} + \overline{B}) C D + (\overline{A} + \overline{B}) (\overline{C} D) + \overline{A} + \overline{B}$$

factor and simplify:

$$\rightarrow Y = (\overline{A} + \overline{B}) (C D + \overline{C} D) + \overline{A} + \overline{B}$$

De Morgan's Law in Reverse:

$$Y = (\overline{A} + \overline{B}) (C D + \overline{C} D) + \overline{A} + \overline{B}$$

$$Y = \overline{A} + \overline{B} \rightarrow AB$$

DeMorgan's

$$Y = \overline{A} \overline{B} + AB$$

$$Y = 1$$

SimplificationPrelab2.PNG

The left image above, shows the corresponding truth table that shows the original logic expression and its truth table that has been found for the following input combinations, although it isn't all of the combinations of inputs, it shows that the output is independent of the inputs and remains 1 no matter what. The right image shows the simplification process of the original expression.

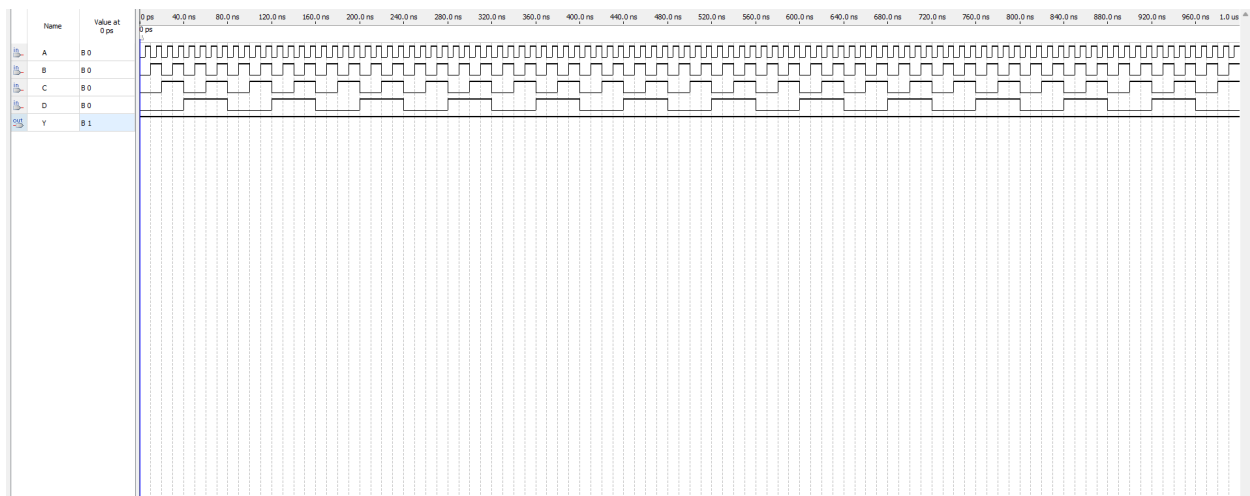
Ultimately the simplified logic was found to be:

$$Y = \overline{AB} + AB$$

Which can be further simplified to just

$$Y = 1$$

For modeling the simplified version however, $Y = \overline{AB} + AB$ would be better to model and understand.



Waveform of original Y expression. (inputs were varied simultaneously during simulation.)

Conclusion: No matter what the inputs were, the output always remains high. Matching the truth table to perfection, confirming that the theoretical truth table was correct and accurate.

Equipment

Hardware

- **Altera DE2-115 Development and Education Board** Used to implement and test combinational logic circuits Toggle switches used as logic inputs LEDs used as logic outputs

- USB-Blaster Programming Cable

- Used to program the FPGA on the DE2-115 board

Software:

- **Intel Quartus Prime**

- Used to design logic circuits using schematics
- Used to compile, simulate, and program.

Logic Components:

- AND gates
- OR gates
- NOT gates
- NAND gates

Input and Output Devices:

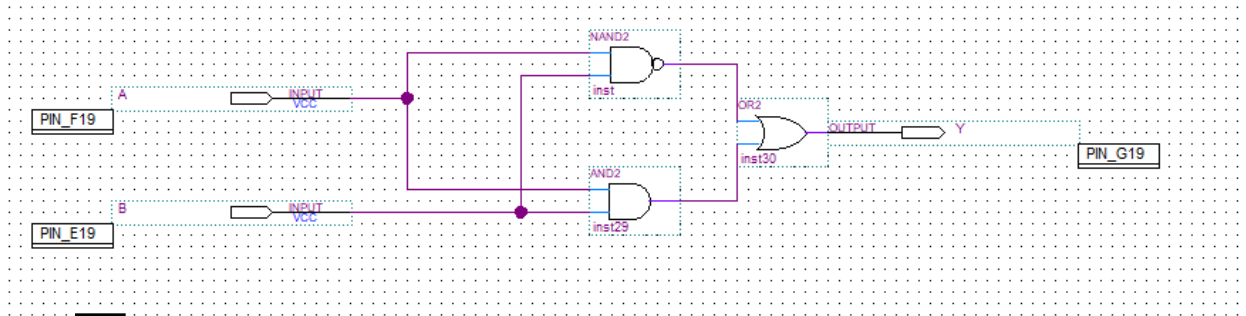
- Toggle switches on the DE2-115 board for input variables (A, B, C, D)
- LEDs on the DE2-115 board for displaying output logic states

Additional Equipment

- Laboratory computer capable of running Quartus
- USB cable for board connection

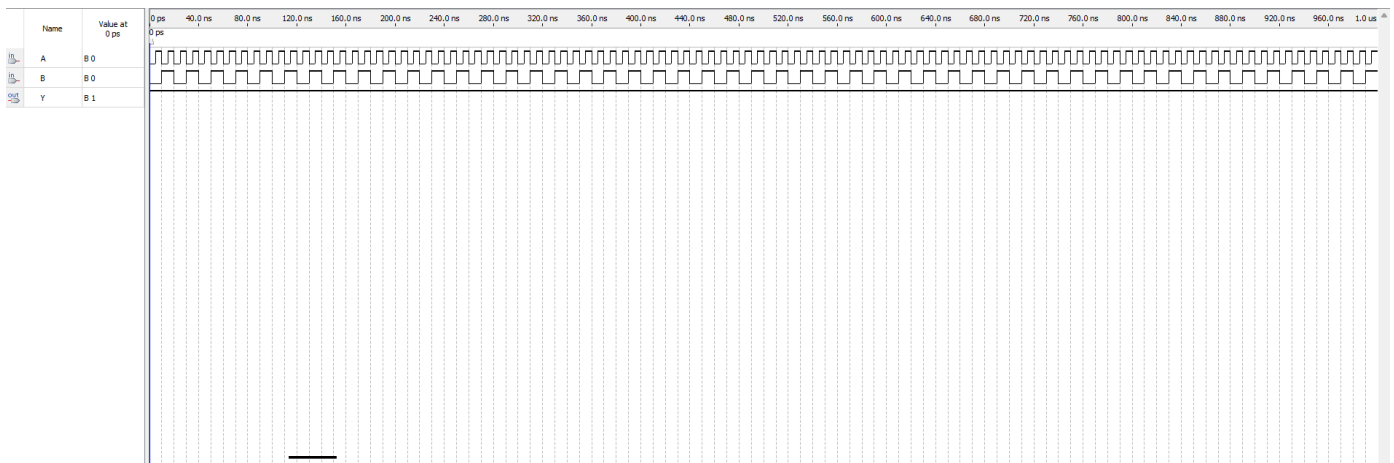
Design Procedure

The purpose of this experiment was to now use the previously determined simplified expression of the output Y's logic expression and recreate a simplified circuit, compare results to experimentally determined truth tables and verifying that it matches the truth table. Lastly assign pins to inputs like push buttons and switches and observe the behavior of the output when the switches are played around with on the Altera board.



$Y = \overline{AB} + AB$ (simplified version of original).

As expected the circuit is much shorter and more simplified compared to the original. It works by taking inputs A and B initially by connecting them to a NAND2 gate which is the first input to the final OR2 and connecting A and B to an AND2 which forms the second input of the final OR2 which connects to the output Y. Listed also next to each input are the specific PIN locations where it is going to be connected to on the Altera board.



Waveform of $Y = \overline{AB} + AB$ (inputs were varied simultaneously during simulation.)

Circuit Logic:

- When A = 1 B = 1, output remains high.
- When A = 1 B = 0, output remains high.
- When A = 0 B = 1, output remains high.
- When A = 0 B = 0, output remains high.

Conclusion: No matter what value the inputs have, the output always remains high (or 1).

A	B	Y	$Y = \overline{A}B + AB$
1	1	1	$\rightarrow Y = \overline{(1 \cdot 1)} + 1 = 0 + 1 = 1$
1	0	1	$\rightarrow Y = \overline{0 \cdot 1} + 0 = 1 + 0 = 1$
0	1	1	$\rightarrow \dots$
0	0	1	$\rightarrow \dots$

TruthTableInlab2.PNG

As shown by the truth table, it perfectly matches the waveform file of the same logic circuit. Therefore confirming the credibility of this truth table compared to an experimental simulation waveform. This shows that the circuits were accurately designed and tested which was further backed up by theoretical findings such as the truth tables. No unexpected outcomes were observed and the smooth flat line expected was observed in both versions of the circuit, both in the non-simplified and simplified.

Discussion of Results

The experimental results obtained from both simulation and hardware implementation confirmed the theoretical behavior of the Boolean logic function analyzed in this lab. The truth table derived during the pre-lab phase showed that the output of the given logic expression remained high for all possible input combinations. This result was preserved after applying Boolean algebra identities to simplify the original expression, indicating that the logic function was independent of its input variables.

Waveforms generated in Quartus served as a verification tool for comparing theoretical predictions with the actual behavior. The waveform outputs of both the original and simplified logic expressions matched the derived truth tables exactly, showing a constant high output throughout all input transitions. This confirmed that the Boolean simplification process did not alter the functional behavior of the circuit, but instead reduced its complexity.

Overall, there was excellent agreement in terms of assessment, simulation waveforms, and hardware testing results, which demonstrated the viability of Boolean algebra as a predictive tool. There were no unforeseen discrepancies, and it was evident that logic simplification was essential in digital circuit design.

Conclusion

This lab focused on the analysis, simplification, and implementation of a Boolean logic function using Boolean algebra and combinational circuit design. By deriving the truth table of the original logic expression and applying Boolean identities, the logic function was successfully simplified while having an identical output. Simulation results obtained through Quartus confirmed that the circuit produced outputs that matched the theoretical predictions for all input combinations.

The experimental results showed no differences between the expected theoretical behavior and the simulated outcomes. Both the original and simplified implementations consistently produced a constant high output, confirming the accuracy of the truth table and the correctness of the Boolean simplification process. The waveform simulations and physical testing on the Altera board further reinforced the reliability of Boolean algebra as a tool for predicting and validating circuit behavior.

Appendices

Appendix A - Prelab

- Lab2Prelab.qsf
- Lab2Prelab.qws
- Waveform.vwf
- Lab2Prelab.bdf
- Lab2PrelabSchematic.PNG
- Lab2PrelabWaveform.PNG
- Lab2Prelab.qpf
- TruthTablePrelab2.PNG
- SimplificationPrelab2.PNG

Appendix B - In Lab

- Inlab2.qsf
- Inlab2.qws
- Inlab2.qpf
- WaveformInlab.vwf
- Inlab2Schematic.PNG
- Inlab2Waveform.PNG
- TruthTableInlab2.PNG

Appendix C – Simulation and Verification Files

- ● Quartus Simulation Waveform Editor files (.vwf)
- ● Waveform and VCD files used for functional verification
- ● Screenshots of simulation results used for comparison with truth tables

Appendix D – Software Environment

- ● Intel Quartus
- ● Quartus Block Diagram/Schematic Editor
- ● Quartus Simulation Waveform Editor